

程式碼實驗室：使用 Neo4j 和 Vertex AI 建構電影推薦聊天機器人

來源：<https://codelabs.developers.google.com/neo4j-vertexai-movie-recommender-python?hl=zh-tw>

步驟數：10

本程式碼研究室會逐步引導您使用 GraphRAG 方法，建構以 GenAI 為技術基礎的電影推薦聊天機器人。您將瞭解如何使用 Neo4j 建立知識圖譜、使用 Vertex AI 的文字嵌入模型產生向量嵌入，以及建構 Gemini 支援的對話式聊天機器人。這個聊天機器人會結合語意搜尋和 LLM 產生的 Cypher 查詢，對圖表進行更深入的推理，並根據自然語言輸入內容，傳回可解釋的相關電影推薦內容。

目錄

1. [1. 總覽](#)
2. [2. 設定 Neo4j AuraDB](#)
3. [3. 事前準備](#)
4. [4. 準備 Movies 資料集](#)
5. [5. 建立電影知識圖譜](#)
6. [6. 生成及載入嵌入項目，執行向量相似度搜尋](#)
7. [7. 電影推薦聊天機器人](#)
8. [8. \(選用\) 部署至 Google Cloud Run](#)
9. [9. 清除所用資源](#)
10. [10. 恭喜](#)

1. 總覽

在本程式碼研究室中，您將結合 Neo4j、Google Vertex AI 和 Gemini 的強大功能，建構智慧型電影推薦聊天機器人。這個系統的核心是 Neo4j 知識圖譜，透過豐富的互連節點和關係網路，模擬電影、演員、導演、類型等。

為提升語意理解能力，進而改善使用者體驗，您將使用 Vertex AI 的 `text-embedding-004` 模型 (或更新版本)，從電影劇情簡介生成向量嵌入。這些嵌入內容會在 Neo4j 中建立索引，以便根據相似度快速檢索。

最後，您將整合 Gemini，打造對話式介面，讓使用者以自然語言提問，例如「如果我喜歡《星際效應》，應該看什麼？」，並根據語意相似度和圖表式背景資訊，獲得個人化電影建議。

在本程式碼研究室中，您將逐步完成下列步驟：

1. 使用電影相關實體和關係建立 Neo4j 知識圖譜
2. 使用 Vertex AI 生成/載入電影簡介的文字嵌入
3. 導入由 Gemini 支援的 Gradio 聊天機器人介面，結合向量搜尋和以圖表為基礎的 Cypher 執行作業
4. (選用) 將應用程式部署至 Cloud Run，做為獨立網頁應用程式

課程內容

- 如何使用 Cypher 和 Neo4j 建立及填入電影知識圖譜
- 如何使用 Vertex AI 生成及處理語意文字嵌入
- 如何結合 LLM 和知識圖譜，使用 GraphRAG 進行智慧型檢索
- 如何使用 Gradio 建構簡單好用的對話介面
- 如何選擇在 Google Cloud Run 上部署

軟硬體需求

- Chrome 網路瀏覽器
- Gmail 帳戶
- 已啟用計費功能的 Google Cloud 專案
- 免費的 Neo4j Aura DB 帳戶
- 對終端機指令和 Python 有基本瞭解 (有幫助，但非必要)

本程式碼實驗室適合各種程度的開發人員 (包括初學者)，並在範例應用程式中使用 Python 和 Neo4j。雖然對 Python 和圖形資料庫有基本瞭解會很有幫助，但您不需要有相關經驗，也能瞭解概念或跟著操作。

Smart Movie Recommender with GraphRAG

Discover movies you'll love — powered by Neo4j and Vertex AI! This assistant combines semantic search with knowledge graph reasoning — using vector similarity for relevant matches and LLM-generated Cypher queries for deeper insights from movie plots, genres, and relationships.

Your movie preference

What kind of movie would you like to watch?

Clear

Submit

Recommendations

Examples

Which time travel movies star Bruce Willis?

Find romantic comedies directed by female directors.

Recommend sci-fi movies featuring AI and starring Keanu Reeves.

Show me thrillers from the 2000s with mind-bending plots.

List superhero movies where the villain turns good.

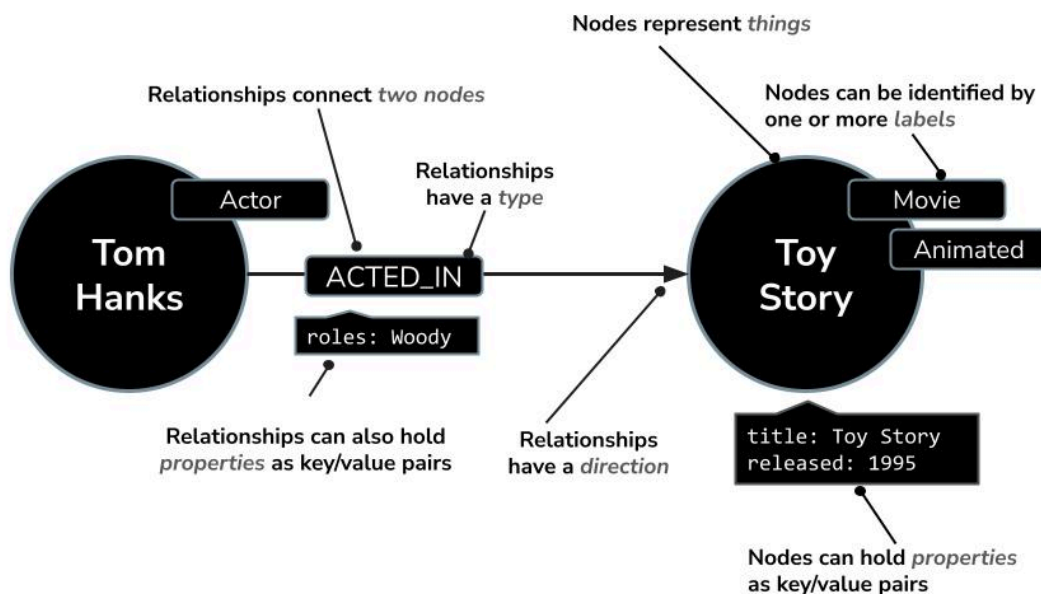
2. 設定 Neo4j AuraDB

Neo4j 是領先的原生圖形資料庫，可將資料儲存為節點 (實體) 和關係 (實體之間的連結) 網路，因此非常適合用於需要瞭解連結的應用情境，例如推薦、詐欺偵測、知識圖譜等。與依賴嚴格資料表或階層式結構的關聯式或文件式資料庫不同，Neo4j 的彈性圖形模型可直覺且有效率地呈現複雜的互連資料。

Neo4j 不像關聯式資料庫那樣以資料列和資料表整理資料，而是使用**圖形模型**，將資訊表示為**節點** (實體) 和**關係** (實體之間的連結)。這個模型非常適合處理本質上相關聯的資料，例如人物、地點、產品，或以我們的例子來說，就是電影、演員和類型。

舉例來說，在電影資料集中：

- 節點可以代表 `Movie`、`Actor` 或 `Director`
- 關係可以是 `ACTED_IN` 或 `DIRECTED`



這種結構可讓您輕鬆提出下列問題：

- 這位演員演過哪些電影？
- 誰曾與克里斯多福諾蘭合作？
- 根據共同演員或類型，找出類似的電影。

Neo4j 內建功能強大的查詢語言「**Cypher**」，專為查詢圖形而設計。Cypher 可讓您以簡潔易讀的方式表達複雜的模式和連結。舉例來說，這項 Cypher 查詢會使用 `MERGE`，確保演員、電影和角色詳細資料的關係是獨一無二，避免重複。

```
MERGE (a:Actor {name: "Tom Hanks"})
MERGE (m:Movie {title: "Toy Story", released: 1995})
MERGE (a)-[:ACTED_IN {roles: ["Woody"]}]->(m);
```

Neo4j 提供多種部署選項，您可以視需求選擇：

- **自行管理**：使用 **Neo4j Desktop** 或 Docker 映像檔 (地端部署或自有雲端) 在自有基礎架構上執行 Neo4j。
- **雲端管理**：使用 Marketplace 產品，在熱門雲端供應商上部署 Neo4j。

- **全代管**：使用 **Neo4j AuraDB**，這是 Neo4j 的全代管雲端資料庫即服務，可為您處理佈建、資源調度、備份和安全防護相關工作。

在本程式碼研究室中，我們將使用 **Neo4j AuraDB Free**，這是 AuraDB 的零成本層級。這個服務提供全代管的圖形資料庫執行個體，具備足夠的儲存空間和功能，可供原型設計、學習和建構小型應用程式，非常適合我們建構以生成式 AI 技術為基礎的電影推薦聊天機器人的目標。

您將建立免費的 AuraDB 執行個體，使用連線憑證將其連線至應用程式，並在本實驗室中，使用該執行個體儲存及查詢電影知識圖譜。

為什麼要使用圖表？

在傳統關聯式資料庫中，如要回答「哪些電影的演員或類型與《全面啟動》相似？」這類問題，**需要跨多個資料表執行複雜的 JOIN 作業**。隨著關係深度增加，效能和可讀性會降低。

不過，Neo4j 等圖形資料庫的設計宗旨是**有效遍歷關係**，因此非常適合用於推薦系統、語意搜尋和智慧助理。這類模型有助於擷取傳統資料模型難以呈現的現實世界情境，例如協作網路、故事情節或觀眾偏好。

將連結的資料與 **Gemini 等 LLM** 和 **Vertex AI 的向量嵌入** 結合，就能強化聊天機器人體驗，讓聊天機器人以更貼近個人需求且相關的方式推理、擷取及回覆。

建立 Neo4j AuraDB Free

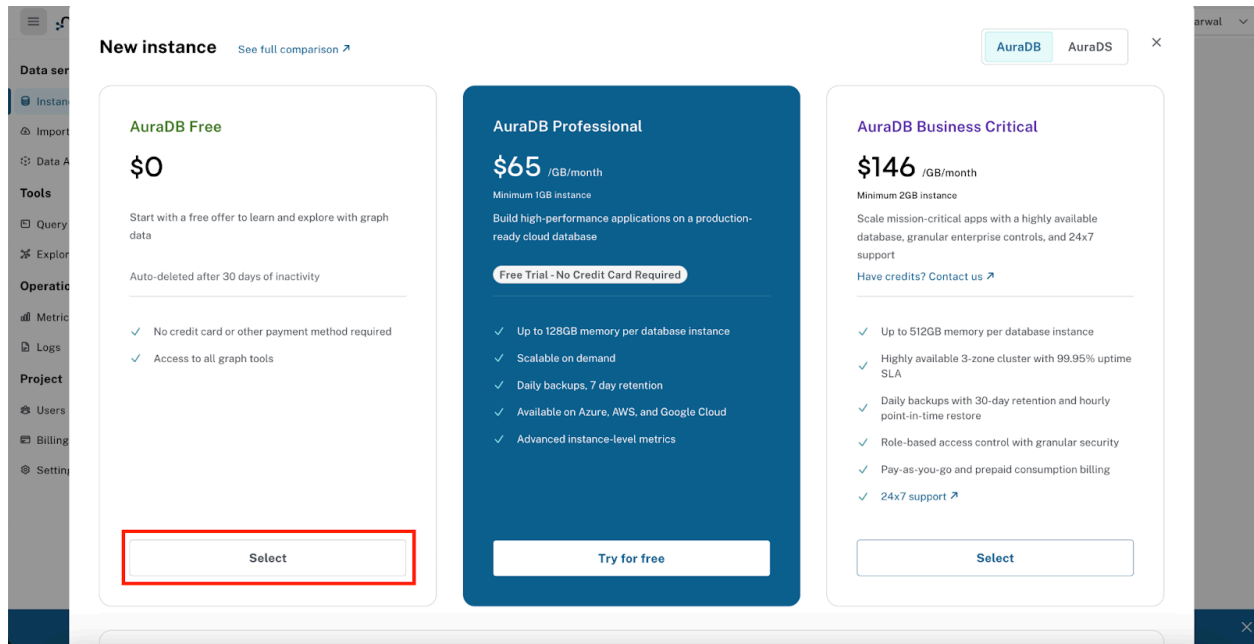
1. 前往 <https://console.neo4j.io>
2. 使用 Google 帳戶或電子郵件登入。
3. 按一下「建立免費執行個體」。
4. 在佈建執行個體時，系統會顯示彈出式視窗，內含資料庫的連線憑證。

請務必從彈出式視窗下載並安全儲存下列詳細資料，這些資料是將應用程式連線至 Neo4j 的必要資訊：

```
NEO4J_URI=neo4j+s://<your-instance-id>.databases.neo4j.io
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=<your-generated-password>
AURA_INSTANCEID=<your-instance-id>
AURA_INSTANCENAME=<your-instance-name>
```

在下一個步驟中，您將使用這些值設定專案中的 .env 檔案，以便向 Neo4j 進行驗證。

⚠ 注意：系統不會自動下載這些憑證。您必須在彈出式視窗顯示時手動下載並儲存。如果忘記密碼，就必須刪除執行個體並建立新的執行個體。因此建議您立即保留安全副本。



Neo4j AuraDB Free 非常適合開發、實驗，以及本程式碼研究室這類的小型應用程式。並提供充足的用量限制，最多可支援 **20 萬個節點**和 **40 萬個關係**。雖然這項服務提供建構及查詢知識圖譜所需的所有基本功能，但並不支援進階設定，例如自訂外掛程式或增加儲存空間。如要處理生產工作負載或較大的資料集，可以升級至更高階的 AuraDB 方案，取得更大的容量、效能和企業級功能。

這樣就完成設定 Neo4j AuraDB 後端的章節。在下一個步驟中，我們將建立 Google Cloud 專案、複製存放區，並設定必要的環境變數，準備好開發環境，然後再開始進程式碼研究室。

3. 事前準備

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，預先載入了 bq。點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 透過下列指令啟用必要的 API。這可能需要幾分鐘的時間，請耐心等待。

```
gcloud services enable cloudresourcemanager.googleapis.com \
    servicenetworking.googleapis.com \
    run.googleapis.com \
    cloudbuild.googleapis.com \
    cloudfunctions.googleapis.com \
    aiplatform.googleapis.com
```

指令執行成功後，畫面上應該會顯示「**Operation finished successfully**」訊息。

除了使用 gcloud 指令，您也可以透過控制台搜尋各項產品，或使用這個[連結](#)。

如果遺漏任何 API，您隨時可以在導入過程中啟用。

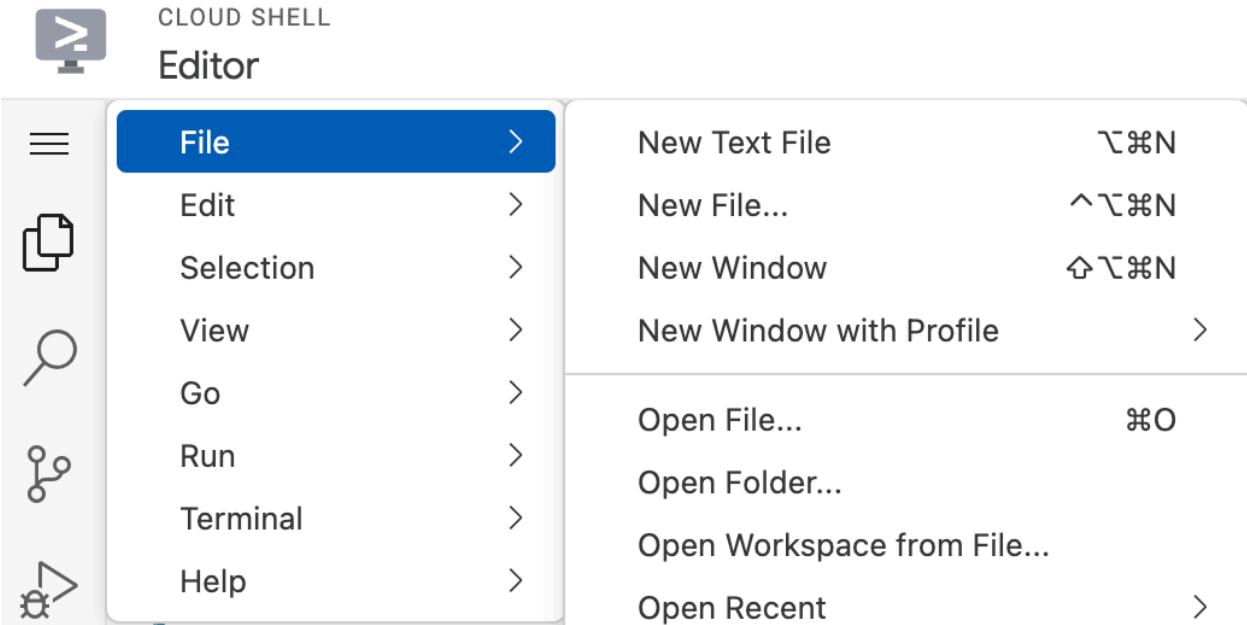
如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

複製存放區並設定環境設定

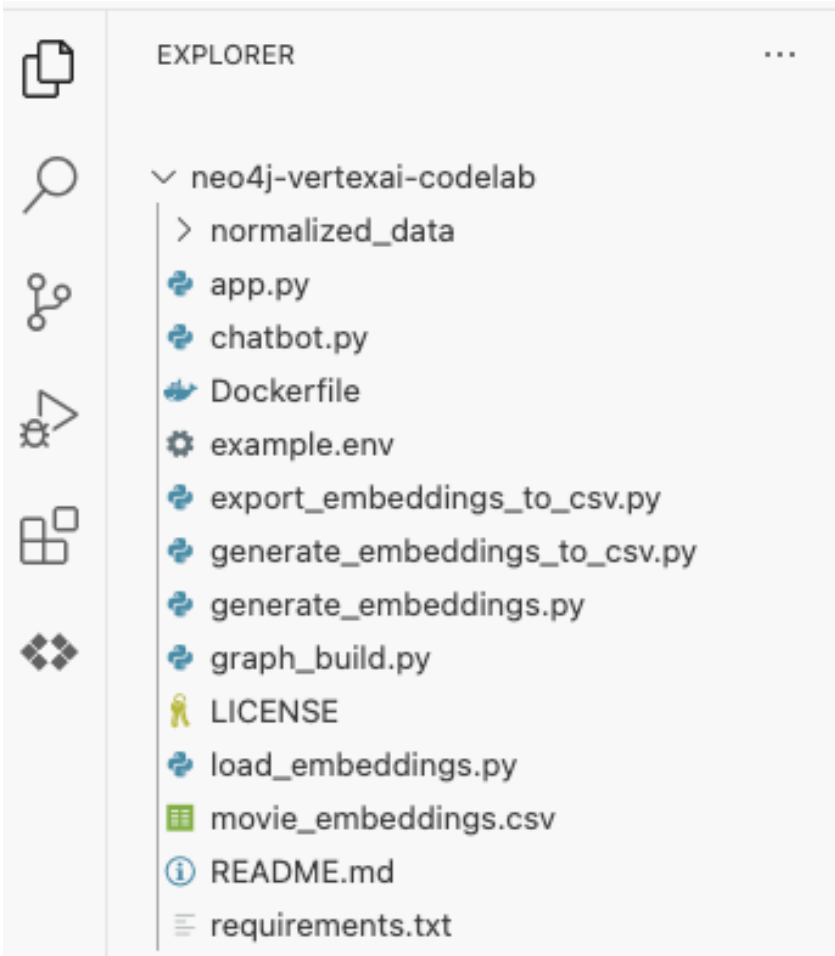
下一步是複製範例存放區，我們會在程式碼研究室的其餘部分參照這個存放區。假設您位於 Cloud Shell 中，請從主目錄執行下列指令：

```
git clone https://github.com/sidagarwal04/neo4j-vertexai-codelab.git
```


如要啟動編輯器，請點選 Cloud Shell 視窗工具列中的「開啟編輯器」。按一下左上角的選單列，然後選取「檔案」→「開啟資料夾」，如下所示：



選取 `neo4j-vertexai-codelab` 資料夾，您應該會看到開啟的資料夾，結構與下圖所示類似：



接著，我們需要設定本程式碼研究室中會用到的環境變數。按一下 `example.env` 檔案，您應該會看到如下所示的內容：

```
NEO4J_URI=  
NEO4J_USER=  
NEO4J_PASSWORD=  
NEO4J_DATABASE=  
PROJECT_ID=  
LOCATION=
```

現在請在與 `example.env` 檔案相同的資料夾中，建立名為 `.env` 的新檔案，並複製現有 `example.env` 檔案的內容。現在，請更新下列變數：

- `NEO4J_URI`、`NEO4J_USER`、`NEO4J_PASSWORD` 和 `NEO4J_DATABASE`：
- 使用上一步建立 Neo4j AuraDB Free 執行個體時提供的憑證，填入這些值。
- `NEO4J_DATABASE` 通常會針對 AuraDB Free 設為 `neo4j`。
- `PROJECT_ID` 和 `LOCATION`：
- 如果您是透過 Google Cloud Shell 執行本程式碼研究室，可以將這些欄位留白，系統會從有效專案設定自動推斷這些值。
- 如果您是在本機或 Cloud Shell 以外的位置執行，請將 `PROJECT_ID` 更新為您先前建立的 Google Cloud 雲端專案 ID，並將 `LOCATION` 設為您為該專案選取的區域 (例如 `us-central1`)。

填寫完這些值後，請儲存 `.env` 檔案。這項設定可讓應用程式連線至 Neo4j 和 Vertex AI 服務。

設定開發環境的最後一個步驟是建立 Python 虛擬環境，並安裝 `requirements.txt` 檔案中列出的所有必要依附元件。這些依附元件包括使用 Neo4j、Vertex AI、Gradio 等所需的程式庫。

首先，請執行下列指令，建立名為 `.venv` 的虛擬環境：

```
python -m venv .venv
```

環境建立完成後，請執行下列指令來啟用建立的環境

```
source .venv/bin/activate
```

現在終端機提示詞開頭應會顯示 `(.venv)`，表示環境已啟動。例如：`(.venv) yourusername@cloudshell:`

現在，請執行下列指令來安裝必要依附元件：

```
pip install -r requirements.txt
```

以下是檔案中列出的主要依附元件快照：

```
gradio>=4.0.0  
neo4j>=5.0.0  
numpy>=1.20.0  
python-dotenv>=1.0.0  
google-cloud-aiplatform>=1.30.0  
vertexai>=0.0.1
```

成功安裝所有依附元件後，本程式碼研究室的指令碼和聊天機器人就能在本地 Python 環境中執行。

太棒了！現在我們已準備好進行下一個步驟：瞭解資料集，並準備好建立圖表和語意擴充功能。

4. 準備 Movies 資料集

我們的首要任務是準備電影資料集，用於建構知識圖表，並為推薦聊天機器人提供支援。我們不會從頭開始，而是使用現有的開放資料集，並以此為基礎建構模型。

我們使用的是 Rounak Banik 的 [電影資料集](#)，這是 Kaggle 上知名的公開資料集。當中包含來自 TMDb 的 45,000 多部電影中繼資料，包括演員、工作人員、關鍵字、分級等。

The Movies Dataset

Metadata on over 45,000 movies. 26 million ratings from over 270,000 users.

[Data Card](#) [Code \(686\)](#) [Discussion \(41\)](#) [Suggestions \(2\)](#)

About Dataset

Context

These files contain metadata for all 45,000 movies listed in the Full MovieLens Dataset. The dataset consists of movies released on or before July 2017. Data points include cast, crew, plot keywords, budget, revenue, posters, release dates, languages, production companies, countries, TMDb vote counts and vote averages.

This dataset also has files containing 26 million ratings from 270,000 users for all 45,000 movies. Ratings are on a scale of 1-5 and have been obtained from the official GroupLens website.

Content

This dataset consists of the following files:

movies_metadata.csv: The main Movies Metadata file. Contains information on 45,000 movies featured in the Full MovieLens dataset. Features include posters, backdrops, budget, revenue, release dates, languages, production countries and companies.

keywords.csv: Contains the movie plot keywords for our MovieLens movies. Available in the form of a stringified JSON Object.

credits.csv: Consists of Cast and Crew Information for all our movies. Available in the form of a stringified JSON Object.

links.csv: The file that contains the TMDb and IMDB IDs of all the movies featured in the Full MovieLens dataset.

links_small.csv: Contains the TMDb and IMDB IDs of a small subset of 9,000 movies of the Full Dataset.

ratings_small.csv: The subset of 100,000 ratings from 700 users on 9,000 movies.

The Full MovieLens Dataset consisting of 26 million ratings and 750,000 tag applications from 270,000 users on all the 45,000 movies in this dataset can be accessed [here](#)

Usability

8.24

License

CC0: Public Domain

Expected update frequency

Not specified

Tags

[Earth and Nature](#)[Movies and TV Shows](#)[Popular Culture](#)

如要建構可靠且有效的電影推薦聊天機器人，請務必從乾淨、一致且結構化資料著手。Kaggle 的 [電影資料集](#) 是豐富的資源，包含超過 45,000 筆電影記錄和詳細的中繼資料，包括類型、演員、工作人員等，但其中也含有雜訊、不一致和巢狀資料結構，不適合用於圖形模型或語意嵌入。

為解決這個問題，我們預先處理並正規化資料集，確保資料集適合建構 Neo4j 知識圖譜，並產生高品質的嵌入內容。這項程序包括：

- 移除重複和不完整的記錄
- 標準化重要欄位 (例如類型名稱、人名)
- 將複雜的巢狀結構 (例如演員和工作人員) 攤平成結構化 CSV 檔案
- 選取約 12,000 部電影的代表性子集，以符合 Neo4j AuraDB Free 的限制

正規化的高品質資料有助於確保：

- **資料品質：**減少錯誤和不一致，提供更準確的建議
- **查詢效能：**簡化結構可提升擷取速度並減少冗餘
- **嵌入準確度：**乾淨的輸入內容可產生更有意義且符合情境的向量嵌入

您可以在這個 [GitHub 存放區](#) 的 `normalized_data/` 資料夾中存取經過清理和正規化的資料集。這個資料集也會鏡像到 Google Cloud Storage bucket，方便在後續的 Python 指令碼中存取。

資料清理完畢後，我們就可以將資料載入 Neo4j，並開始建構電影知識圖譜。

5. 建立電影知識圖譜

為了支援啟用生成式 AI 的電影推薦聊天機器人，我們需要以特定方式建構電影資料集，擷取電影、演員、導演、類型和其他中繼資料之間豐富的連結網路。在本節中，我們將使用您先前準備的已清除及正規化資料集，在 Neo4j 中建構電影知識圖譜。

我們會使用 Neo4j 的 `LOAD CSV` 功能，擷取公開 Google Cloud Storage (GCS) bucket 中代管的 CSV 檔案。這些檔案代表電影資料集的不同元件，例如電影、類型、演員、工作人員、製作公司和使用者評分。

⚠ 注意：下列步驟旨在協助您瞭解圖表的建構方式。您不需要手動執行這些 **Cypher** 指令。我們強烈建議您使用本節結尾所述的自動化 Python 指令碼，將完整資料集載入 Neo4j。

步驟 1：建立限制和索引

匯入資料前，建議您先建立限制條件和索引，確保資料完整性並提升查詢效能。

```
CREATE CONSTRAINT unique_tmdb_id IF NOT EXISTS FOR (m:Movie) REQUIRE m.tmdbId IS UNIQUE;
CREATE CONSTRAINT unique_movie_id IF NOT EXISTS FOR (m:Movie) REQUIRE m.movieId IS UNIQUE;
CREATE CONSTRAINT unique_prod_id IF NOT EXISTS FOR (p:ProductionCompany) REQUIRE p.company_id
IS UNIQUE;
CREATE CONSTRAINT unique_genre_id IF NOT EXISTS FOR (g:Genre) REQUIRE g.genre_id IS UNIQUE;
CREATE CONSTRAINT unique_lang_id IF NOT EXISTS FOR (l:SpokenLanguage) REQUIRE l.language_code
IS UNIQUE;
CREATE CONSTRAINT unique_country_id IF NOT EXISTS FOR (c:Country) REQUIRE c.country_code IS
UNIQUE;

CREATE INDEX actor_id IF NOT EXISTS FOR (p:Person) ON (p.actor_id);
CREATE INDEX crew_id IF NOT EXISTS FOR (p:Person) ON (p.crew_id);
CREATE INDEX movieId IF NOT EXISTS FOR (m:Movie) ON (m.movieId);
CREATE INDEX user_id IF NOT EXISTS FOR (p:Person) ON (p.user_id);
```

步驟 2：匯入電影中繼資料和關係

接下來，我們將瞭解如何使用 `LOAD CSV` 指令匯入電影中繼資料。這個範例會建立電影節點，並使用名稱、簡介、語言和片長等重要屬性：

```

LOAD CSV WITH HEADERS FROM "https://storage.googleapis.com/neo4j-vertexai-
codelab/normalized_data/normalized_movies.csv" AS row
WITH row, toInteger(row.tmbdId) AS tmbdId
WHERE tmbdId IS NOT NULL
WITH row, tmbdId
LIMIT 12000
MERGE (m:Movie {tmbdId: tmbdId})
ON CREATE SET m.title = coalesce(row.title, "None"),
              m.original_title = coalesce(row.original_title, "None"),
              m.adult = CASE
                          WHEN toInteger(row.adult) = 1 THEN 'Yes'
                          ELSE 'No'
                        END,
              m.budget = toInteger(coalesce(row.budget, 0)),
              m.original_language = coalesce(row.original_language, "None"),
              m.revenue = toInteger(coalesce(row.revenue, 0)),
              m.tagline = coalesce(row.tagline, "None"),
              m.overview = coalesce(row.overview, "None"),
              m.release_date = coalesce(row.release_date, "None"),
              m.runtime = toFloat(coalesce(row.runtime, 0)),
              m.belongs_to_collection = coalesce(row.belongs_to_collection, "None");

```

同樣地，您可以使用各自的 CSV 和 Cypher 查詢，匯入並連結其他實體，例如類型、製作公司、語言、國家/地區、演員、工作人員和使用者評分。

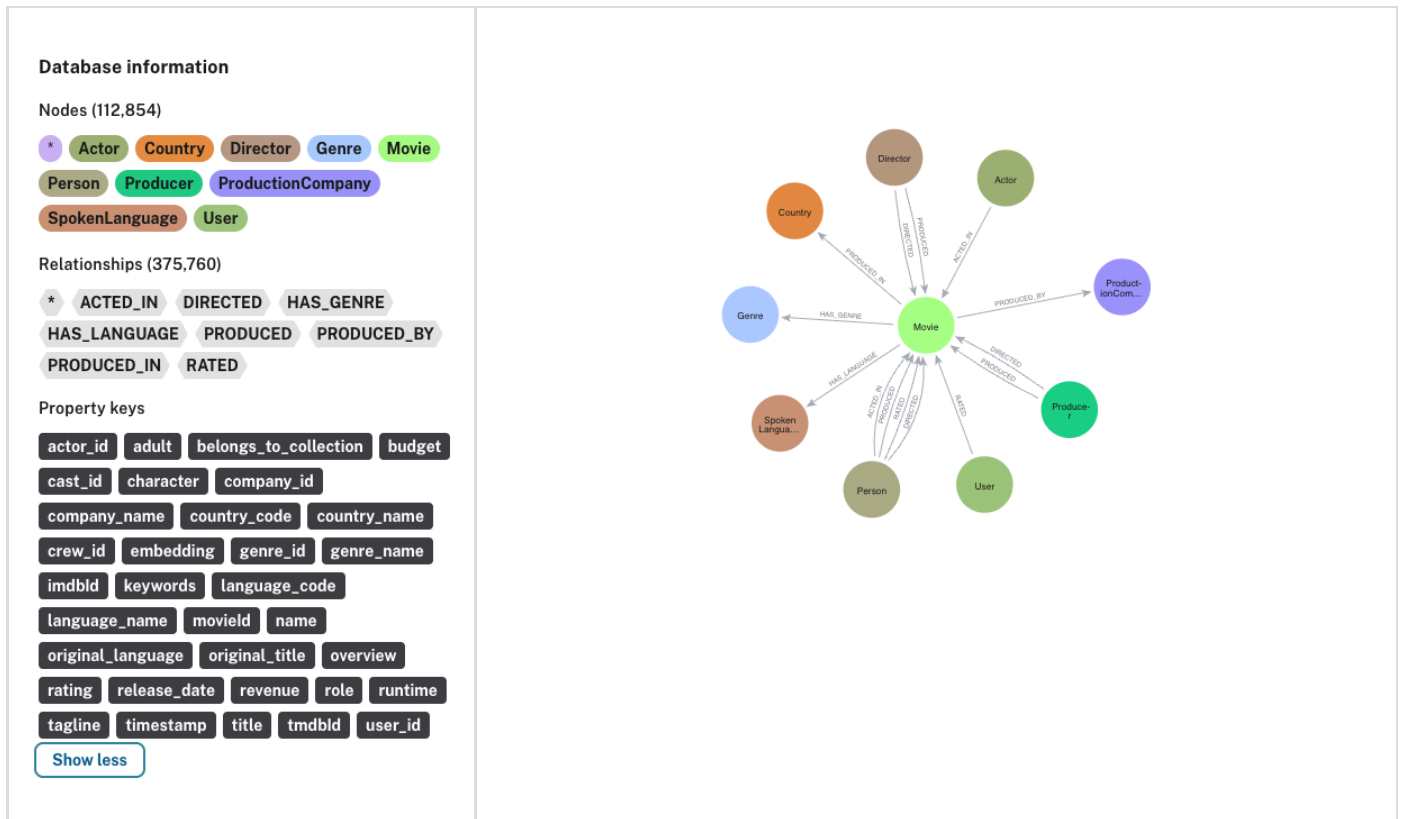
透過 Python 載入完整圖表

建議您使用本程式碼研究室提供的自動 Python 指令碼，不必手動執行多個 Cypher 查詢。

這個指令碼會使用 `.env` 檔案中的憑證，將整個資料集從 GCS 載入至 Neo4j AuraDB 執行個體。 `graph_build.py`

```
python graph_build.py
```

指令碼會依序載入所有必要的 CSV 檔案、建立節點和關係，並建構完整的電影知識圖譜。



驗證圖表

載入後，您可以執行下列指令碼來驗證圖表：

```
python validate_graph.py
```

這會快速摘要說明圖表內容：有多少電影、演員、類型和關係 (例如 ACTED_IN、DIRECTED 等)，確保匯入成功。

 Node Counts:

Movie: 11997 nodes
ProductionCompany: 7961 nodes
Genre: 20 nodes
SpokenLanguage: 100 nodes
Country: 113 nodes
Person: 92663 nodes
Actor: 81165 nodes
Director: 4846 nodes
Producer: 5981 nodes
User: 671 nodes

 Relationship Counts:

HAS_GENRE: 28479 relationships
PRODUCED_BY: 22758 relationships
PRODUCED_IN: 14702 relationships
HAS_LANGUAGE: 16184 relationships
ACTED_IN: 191307 relationships
DIRECTED: 5047 relationships
PRODUCED: 6939 relationships
RATED: 90344 relationships

現在圖表應該會填入電影、人物、類型等資料，準備在下一個步驟中進行語意擴充！

6. 生成及載入嵌入項目，執行向量相似度搜尋

如要在聊天機器人中啟用語意搜尋，我們需要為電影簡介生成向量嵌入。這些嵌入會將文字資料轉換為數值向量，以便比較相似度，因此即使查詢內容與片名或說明不完全相符，聊天機器人也能擷取相關電影。

什麼是嵌入和向量？

您可以將嵌入視為將字詞或句子轉換為一連串的數字 (稱為向量)，藉此掌握其意義。例如：

- 「太空冒險」→ [0.12, 0.88, -0.45, ..., 0.31]
- 「銀河之旅」→ [0.10, 0.85, -0.47, ..., 0.29]

這兩句話看起來不同，但在向量空間中很接近，因此有助於聊天機器人瞭解兩者討論的是類似概念。

⚠ 效能注意事項

即時為數千部電影說明生成嵌入內容，既耗時又耗費資源。為提高本程式碼實驗室的效率和易用性，我們已使用 Vertex AI 的 **text-embedding-004** 模型，預先計算所有約 12,000 部電影簡介的嵌入內容，並將其儲存在公開提供的 CSV 檔案中。

movie_embeddings			
tmdbid	title	overview	embedding
862	Toy Story	Led by Woody, Andy's toys live happily in his room until Andy's	[-0.019403647631406784, -0.01738525927066803, 0.02710539661347866, -0.0184...
8844	Jumanji	When siblings Judy and Peter discover an enchanted board game	[-0.007400821428745985, -0.05172286555171013, 0.06909646838903427, -0.0517...
15602	Grumpier Old Men	A family wedding reignites the ancient feud between next-door	[-0.007951149716973305, 0.00031322980066761374, -0.004160706419497728, 0.0...
31357	Waiting to Exhale	Cheated on, mistreated and stepped on, the women are holding	[-0.016324903815984726, -0.03980735316872597, 0.0014963417779654264, -0.07...
11862	Father of the Bride Part II	Just when George Banks has recovered from his daughter's we	[-0.031691115349531174, -0.023315802216529846, -0.03291140869259834, -0.013...
949	Heat	Obsessive master thief, Neil McCauley leads a top-notch crew	[0.006021352019160986, -0.008987922221422195, -0.004541116300970316, 0.031...
11860	Sabrina	An ugly duckling having undergone a remarkable change, still h	[-0.010569791309535503, -0.03190692886710167, -0.035053662955760956, -0.02...
45325	Tom and Huck	A mischievous young boy, Tom Sawyer, witnesses a murder by	[-0.051339369267225266, -0.02224826253950596, 0.0359366200864315, -0.00236...
9091	Sudden Death	International action superstar Jean Claude Van Damme teams	[0.03026837855577469, -0.0676434263586998, -0.011608698405325413, -0.03198...
710	GoldenEye	James Bond must unmask the mysterious head of the Janus St	[-0.008222734555602074, -0.023640062659978867, -0.01391320675611496, -0.04...
9087	The American President	Widowed U.S. president Andrew Shepherd, one of the world's	[-0.011325136758387089, -0.00020732746634166688, -0.07319273054599762, 0.0...
12110	Dracula: Dead and Loving It	When a lawyer shows up at the vampire's doorstep, he falls pre	[-0.004572826903313398, -0.022336801514029503, 0.00983049813657999, -0.010...
21032	Balto	An outcast half-wolf risks his life to prevent a deadly epidemic	[-0.022955430671572685, -0.0060389358550310135, -0.021964332088828087, -0.0...
10858	Nixon	An all-star cast powers this epic look at American President Ric	[-0.02491925098001957, -0.06040237843990326, -0.0773150622844696, -0.00296...
1408	Cutthroat Island	Morgan Adams and her slave, William Shaw, are on a quest to	[-0.040008097887039185, -0.03223258629441261, 0.020747391507029533, 0.0239...
524	Casino	The life of the gambling paradise - Las Vegas - and its dark ma	[0.023899123072624207, -0.009660971350967884, -0.036351677030324936, -0.02...
4584	Sense and Sensibility	Rich Mr. Dashwood dies, leaving his second wife and her daug	[-0.007593857124447823, -0.025318007916212082, 0.03157714754343033, -0.016...
5	Four Rooms	It's Ted the Bellhop's first night on the job...and the hotel's very	[0.007400691043585539, -0.003934323322027922, 0.0027352042961865664, 0.026...
9273	Ace Ventura: When Nature Calls	Summoned from an ashram in Tibet, Ace finds himself on a per	[0.012106134556233883, -0.007368657737970352, 0.0021846890449523926, -0.05...
11517	Money Train	A vengeful New York transit cop decides to steal a trainload of	[-0.07688142359256744, -0.010417266748845577, 0.04447673633694649, -0.0047...
8012	Get Shorty	Chili Palmer is a Miami mobster who gets sent by his boss, the	[-0.032241374254226685, -0.01298498548567295, -0.013693290762603283, -0.02...
1710	Copycat	An agoraphobic psychologist and a female detective must wor	[-0.005622470751404762, -0.04331270605325699, 0.030909910798072815, -0.011...

選項 1：透過 Cypher 載入預先計算的嵌入內容

如要將嵌入內容快速附加至 Neo4j 中的對應 `Movie` 節點，請在 Neo4j 瀏覽器中執行下列 Cypher 指令：

```
LOAD CSV WITH HEADERS FROM 'https://storage.googleapis.com/neo4j-vertexai-
codelab/movie_embeddings.csv' AS row
WITH row
MATCH (m:Movie {tmdbId: toInteger(row.tmdbId)})
SET m.embedding = apoc.convert.fromJsonList(row.embedding)
```

這項指令會從 CSV 讀取嵌入向量，並將這些向量附加為每個 `Movie` 節點的屬性 (`m.embedding`)。

選項 2：使用 Python 載入嵌入內容

您也可以使用提供的 Python 指令碼，以程式輔助方式載入嵌入內容。如果您是在自己的環境中工作，或是想自動執行這項程序，這個方法就非常實用：

```
python load_embeddings.py
```

這個指令碼會從 GCS 讀取相同的 CSV，並使用 Python Neo4j 驅動程式將嵌入內容寫入 Neo4j。

[選用] 自行產生嵌入 (用於探索)

如要瞭解生成嵌入的過程，可以探索 `generate_embeddings.py` 指令碼中的邏輯。這項功能會使用 Vertex AI，透過 `text-embedding-004` 模型嵌入每部電影的簡介文字。

如要自行嘗試，請開啟並執行程式碼的嵌入生成部分。如果您在 Cloud Shell 中執行，可以註解下列程式碼行，因為 Cloud Shell 已透過有效帳戶完成驗證：

```
# os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = "./service-account.json"
```

⚠ 執行完整的嵌入生成程序會耗用 Vertex AI 配額，且可能需要相當長的時間，尤其是處理數千筆記錄時。

將嵌入項目載入 Neo4j 後，電影知識圖譜就會具備語意感知能力，可使用向量相似度支援強大的自然語言搜尋功能！

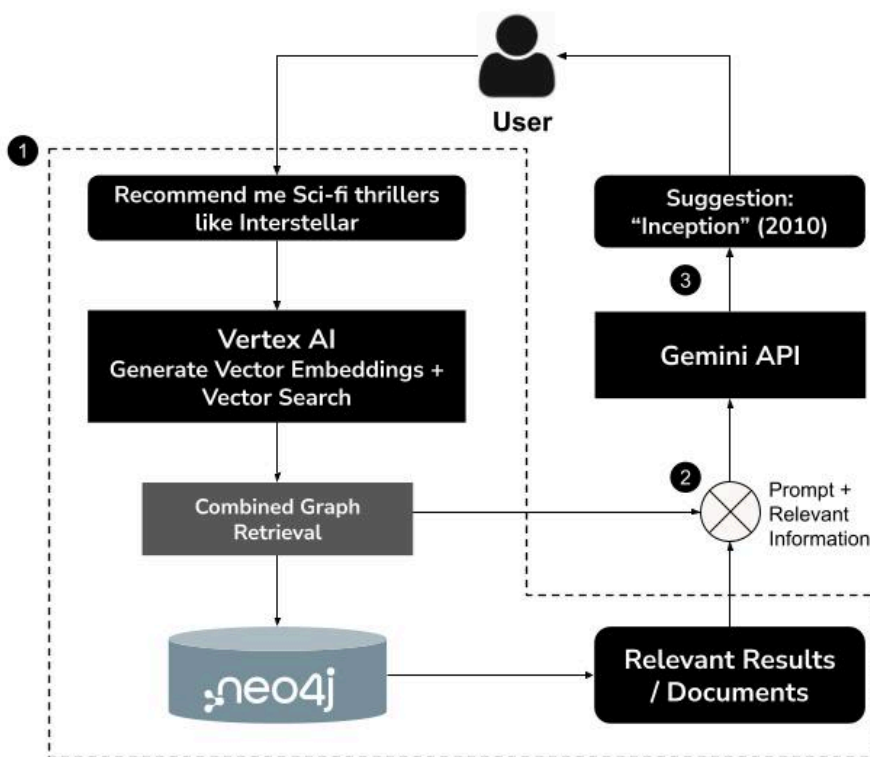
7. 電影推薦聊天機器人

知識圖譜和向量嵌入內容就位後，現在可以將所有內容整合到全功能對話式介面中，也就是以 **GenAI** 技術為基礎的電影推薦聊天機器人。

這個聊天機器人是使用 **Python** 透過 **Gradio** 實作，Gradio 是一種輕量型網頁框架，可建構直覺式使用者介面。核心邏輯位於 `app.py`，可連線至 **Neo4j AuraDB** 執行個體，並使用 **Google Vertex AI** 和 **Gemini** 處理自然語言查詢並做出回應。

運作方式

1. 使用者輸入自然語言查詢例如「推薦類似《星際效應》的科幻驚悚片」
2. 使用 Vertex AI 的 `text-embedding-004` 模型生成查詢的向量嵌入
3. 在 Neo4j 中執行向量搜尋，檢索語意相似的電影
4. 使用 **Gemini** 執行下列操作：
 - 根據背景資訊解讀查詢
 - 根據向量搜尋結果和 Neo4j 結構定義，產生自訂 **Cypher** 查詢
 - 執行查詢，擷取相關圖形資料 (例如演員、導演、類型)
 - 以對話方式為使用者總結結果



這種混合做法稱為 **GraphRAG (圖形檢索增強生成)**，結合語意檢索和結構化推論，可生成更準確、符合情境且可解釋的建議。

在本機執行 Chatbot

啟動虛擬環境 (如果尚未啟動)，然後使用下列指令啟動聊天機器人：

```
python app.py
```

畫面會顯示類似以下的輸出：

```
Vector index 'overview_embeddings' already exists. No need to create a new one.  
* Running on local URL: http://0.0.0.0:8080  
To create a public link, set `share=True` in `launch()`.
```

💡 如要對外共用聊天機器人，請在 `app.py` 的 `launch()` 函式中設定 `share=True`。

與聊天機器人互動

開啟終端機中顯示的本機網址 (通常為 🖱️ <http://0.0.0.0:8080>)，即可存取聊天機器人介面。

你可以問以下問題：

- 「如果我喜歡《星際效應》，應該看什麼？」
- 「推薦諾拉艾芙倫執導的愛情片」
- 「我想看湯姆漢克斯演出的家庭電影」
- 「Find thriller movies involving artificial intelligence」(尋找與人工智慧相關的驚悚片)

Smart Movie Recommender with GraphRAG

Discover movies you'll love — powered by Neo4j and Vertex AI! This assistant combines semantic search with knowledge graph reasoning — using vector similarity for relevant matches and LLM-generated Cypher queries for deeper insights from movie plots, genres, and relationships.

Your movie preference

What kind of movie would you like to watch?

Clear

Submit

Recommendations

☰ Examples

Which time travel movies star Bruce Willis?

Find romantic comedies directed by female directors.

Recommend sci-fi movies featuring AI and starring Keanu Reeves.

Show me thrillers from the 2000s with mind-bending plots.

List superhero movies where the villain turns good.

聊天機器人會：

- ✔ 瞭解查詢內容
- ✔ 使用嵌入功能找出語意相似的電影劇情
- ✔ 生成並執行 Cypher 查詢，擷取相關圖表內容
- ✔ 幾秒內即可傳回友善的個人化建議

目前可用的功能

您剛建構了以 **GraphRAG** 為基礎的電影聊天機器人，結合了以下功能：

- **向量搜尋**，可找出語意相關性
- 使用 Neo4j 進行知識圖譜推論
- 透過 Gemini 使用 **LLM 功能**
- 使用 Gradio 打造流暢的對話介面

這個架構可做為基礎，您可擴充為由生成式 AI 驅動的進階搜尋、推薦或推論系統。

8. (選用) 部署至 Google Cloud Run

Smart Movie Recommender with GraphRAG

Discover movies you'll love — powered by Neo4j and Vertex AI! This assistant combines semantic search with knowledge graph reasoning — using vector similarity for relevant matches and LLM-generated Cypher queries for deeper insights from movie plots, genres, and relationships.

Your movie preference

What kind of movie would you like to watch?

Clear

Submit

Recommendations

Examples

- Which time travel movies star Bruce Willis?
- Find romantic comedies directed by female directors.
- Recommend sci-fi movies featuring AI and starring Keanu Reeves.
- Show me thrillers from the 2000s with mind-bending plots.
- List superhero movies where the villain turns good.

如要公開存取電影推薦聊天機器人，可以將其部署至 Google Cloud Run。這個全代管的無伺服器平台會自動擴充應用程式，並省去所有基礎架構相關問題。

這項部署作業使用：

- `requirements.txt`：定義 Python 依附元件 (Neo4j、Vertex AI、Gradio 等)
- `Dockerfile`：封裝應用程式
- `.env.yaml`：在執行階段安全地傳遞環境變數

步驟 1：準備 `.env.yaml`

在根目錄中建立名為 `.env.yaml` 的檔案，並加入下列內容：

```
NEO4J_URI: "neo4j+s://<your-aura-db-uri>"
NEO4J_USER: "neo4j"
NEO4J_PASSWORD: "<your-password>"
PROJECT_ID: "<your-gcp-project-id>"
LOCATION: "<your-gcp-region>" # e.g. us-central1
```

💡 建議使用這個格式，而非 `--set-env-vars`，因為這個格式更具擴充性、可控管版本，且容易閱讀。

步驟 2：設定環境變數

在終端機中設定下列環境變數 (將預留位置值替換為實際專案設定)：

```
# Set your Google Cloud project ID
export GCP_PROJECT='your-project-id' # Change this

# Set your preferred deployment region
export GCP_REGION='us-central1'
```

步驟 2：建立 Artifact Registry 並建構容器

```
# Artifact Registry repo and service name
export AR_REPO='your-repo-name' # Change this
export SERVICE_NAME='movies-chatbot' # Or any name you prefer

# Create the Artifact Registry repository
gcloud artifacts repositories create "$AR_REPO" \
  --location="$GCP_REGION" \
  --repository-format=Docker

# Authenticate Docker with Artifact Registry
gcloud auth configure-docker "$GCP_REGION-docker.pkg.dev"

# Build and submit the container image
gcloud builds submit \
  --tag "$GCP_REGION-docker.pkg.dev/$GCP_PROJECT/$AR_REPO/$SERVICE_NAME"
```

這個指令會使用 `Dockerfile` 封裝應用程式，並將容器映像檔上傳至 Google Cloud Artifact Registry。

步驟 3：部署至 Cloud Run

現在請使用 `.env.yaml` 檔案進行執行階段設定，然後部署應用程式：

```
gcloud run deploy "$SERVICE_NAME" \
  --port=8080 \
  --image="$GCP_REGION-docker.pkg.dev/$GCP_PROJECT/$AR_REPO/$SERVICE_NAME" \
  --allow-unauthenticated \
  --region=$GCP_REGION \
  --platform=managed \
  --project=$GCP_PROJECT \
  --env-vars-file=.env.yaml
```

存取聊天機器人

部署完成後，Cloud Run 會提供公開網址，例如：

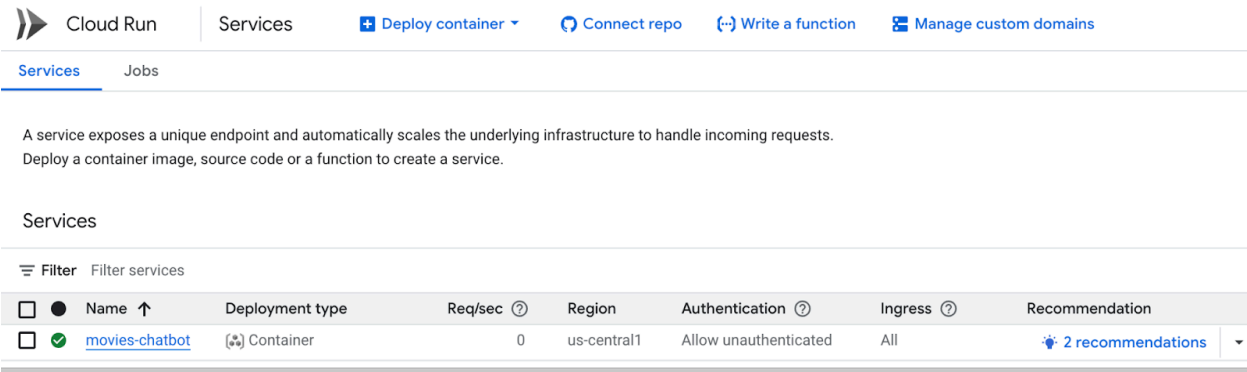
```
https://movies-reco-[UNIQUE_ID].${GCP_REGION}.run.app
```

在瀏覽器中開啟這個網址，即可存取已部署的 Gradio 聊天機器人介面，準備好使用 GraphRAG、Gemini 和 Neo4j 處理電影推薦！

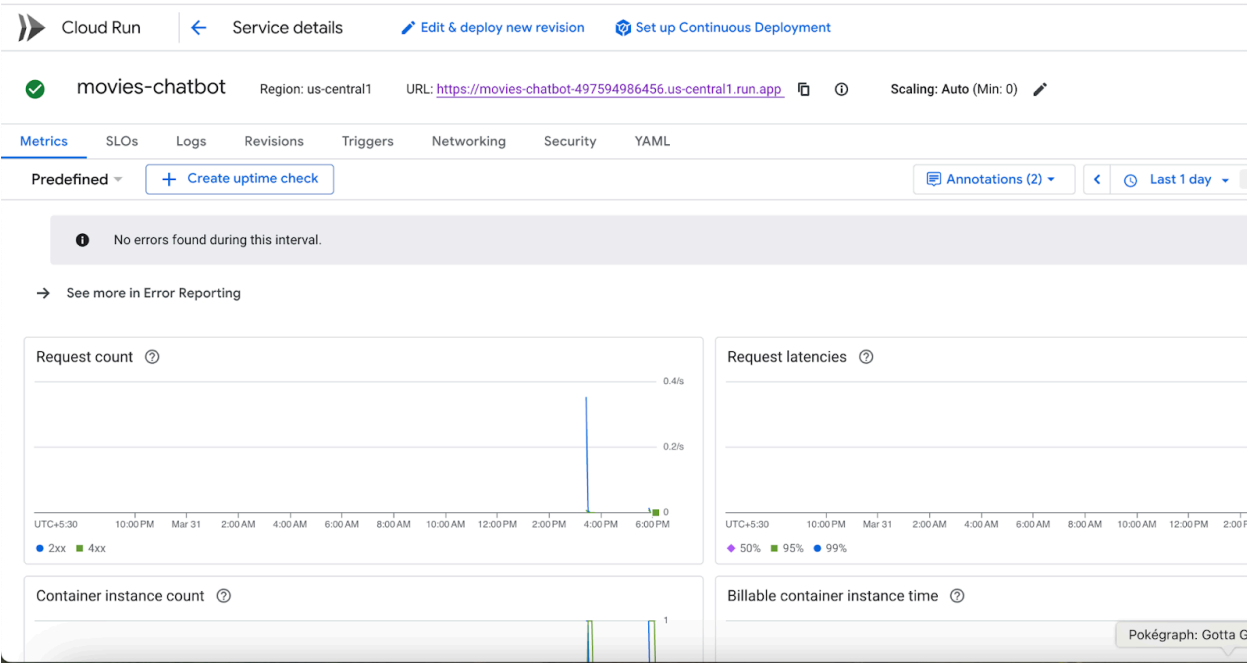
附註和提示

- 確認 `Dockerfile` 在建構期間執行 `pip install -r requirements.txt`。
- 如果未使用 **Cloud Shell**，您需要使用具備 Vertex AI 和 Artifact Registry 權限的服務帳戶，驗證環境。
- 您可以透過 [Google Cloud 控制台 > Cloud Run](#) 監控部署記錄和指標。

您也可以從 Google Cloud 控制台前往 [Cloud Run](#)，查看 Cloud Run 中的服務清單。`movies-chatbot` 服務應是列於該處的服務之一 (如果不是唯一服務)。



按一下特定服務名稱 (在本例中為 `movies-chatbot`)，即可查看服務詳細資料，例如網址、設定、記錄等。



這樣一來，你的電影推薦聊天機器人就部署完成、可擴充且可分享。🎉

步驟 9 · 約 0 分鐘

9. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-

10. 恭喜

您已使用 **Neo4j**、**Vertex AI** 和 **Gemini**，成功建構及部署以 **GraphRAG** 為基礎的生成式 AI 強化電影推薦聊天機器人。結合 Neo4j 的圖形原生建模功能、Vertex AI 的語意搜尋功能，以及 Gemini 的自然語言推論功能，您已建立智慧系統，不僅能執行基本搜尋，還能瞭解使用者意圖、根據連結資料進行推論，以及以對話方式回覆。

在本程式碼研究室中，您完成了下列事項：

- ✅ 在 Neo4j 中建構真實世界的電影知識圖譜，模擬電影、演員、類型和關係
- ✅ 使用 Vertex AI 的文字嵌入模型，為電影劇情簡介生成向量嵌入
- ✅ 導入 GraphRAG，結合向量搜尋和 LLM 生成的 Cypher 查詢，進行更深入的多跳推理
- ✅ 整合 Gemini，解讀使用者問題、生成 Cypher 查詢，並以自然語言歸納圖表結果
- ✅ 使用 Gradio 建立操作直覺的聊天介面
- ✅ 選擇將聊天機器人部署至 Google Cloud Run，享有可擴充的無伺服器主機服務

後續步驟

這項架構不只適用於電影推薦，還可擴展至：

- 書籍和音樂探索平台
- 學術研究助理
- 產品推薦引擎
- 醫療、金融和法律知識助理

只要有複雜關係 + 豐富的文字資料，知識圖譜 + LLM + 語意嵌入的組合就能為新一代智慧型應用程式提供強大動力。

隨著 Gemini 等多模態生成式 AI 模型不斷演進，您將能納入更豐富的背景資訊、圖片、語音和個人化設定，打造真正以人為本的系統。

請繼續探索及建構，別忘了隨時掌握 **Neo4j**、**Vertex AI** 和 **Google Cloud** 的最新消息，讓智慧型應用程式更上一層樓！
如要瞭解更多知識圖譜實作教學課程，請前往 [Neo4j GraphAcademy](#)。

參考文件

- [什麼是 GraphRAG？](#)
 - [Neo4j GenAI 生態系統](#)
 - [Neo4j 向量索引和搜尋](#)
 - [Vertex AI 中的文字嵌入](#)
 - [Cloud Run](#)
-